Hi,

It's time to enhance stateful processing and provide real-time business insight to our users!

Exercise 1 - stateless into stateful conversion

Scala/Java

1. First, create a new Kafka input DataFrame:

```
val inputStream = sparkSession.readStream.format("kafka")
 .options(Map(
   "kafka.bootstrap.servers" -> "localhost:29092",
   "subscribe" -> InputTopic,
   "startingOffsets" -> "EARLIEST"
 )).load()
```

2. Read the input data:

```
val inputVisits = inputStream.selectExpr("CAST(value AS STRING) AS value",
"timestamp")
 .select(functions.from_json($"value".cast("string"),
        UserVisitWithTimestamp.SchemaAsStructType).as("visit"),
   $"timestamp")
 .selectExpr("visit.*", "timestamp")
 .as[UserVisitWithTimestamp]
```

** I'm using the Apache Kafka append time in my idempotency strategy; hence you can see the value and timestamp in the deserialized columns **

Besides, the UserVisitWithTimestamp contains all the columns read from the input.

3. Keep the filtering logic and prepare the dataset for the stateful transformation:

```
val validVisits = inputVisits.filter(visit => {
 visit.visitedPage != null && visit.eventTime != null
})
val sessionTimeout = TimeUnit.MINUTES.toMillis(16)
val userVisits: KeyValueGroupedDataset[Int, UserVisitWithTimestamp] = validVisits
 .withWatermark("eventTime", "15 minutes")
 .groupByKey(_.userId)
```

4. Call your stateful mapping function:

```
val userSessions: Dataset[Option[VisitOutput]] = userVisits.mapGroupsWithState(
 GroupStateTimeout.EventTimeTimeout()
)(
 VisitsMapper.generateVisitDuration(sessionTimeout)
)
```

5. Read the reference dataset for the browser information:

```
val technicalReferenceDataset = sparkSession.read
 .schema("id STRING, name STRING, version STRING")
 .json(ReferenceDatasetFileName)
```

6. Process the userSessions by filtering out the empty rows and joining them with the reference dataset. You can also prepare the final output at this moment by using the TO_JSON function:

```
val sessionsToWrite = userSessions.filter(session => session.isDefined)
 .map(session => session.get)
 .join(technicalReferenceDataset,
   functions.expr("""
     |id = browserCode AND version = browserVersion
     |""".stripMargin), "left_outer")
 .withColumn("technicalContext", functions.struct($"name", $"version"))
 .drop("browserCode", "browserVersion", "id", "name", "version")
 .selectExpr("TO_JSON(STRUCT(*)) AS value")
```

7. Write the transformation result to the output Kafka topic:

```
val writeQuery = sessionsToWrite.writeStream
 .outputMode(OutputMode.Update())
 .format("kafka")
 .options(
   Map(
     "checkpointLocation" -> "/tmp/bde/6/e1/checkpoint/stateful",
     "kafka.bootstrap.servers" -> "localhost:29092",
     "topic" -> OutputTopic
   )
 )
 .start()
```

Python

1. Include Apache Kafka dependency in the SparkSession

```
spark_session = SparkSession.builder.master("local[*]") \
   .config('spark.jars.packages',
'org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0') \
   .config("spark.sql.shuffle.partitions", 2).getOrCreate()
```

2. First, create a new Kafka input DataFrame:

```
input_data = spark_session.readStream.format("kafka") \
   .option("kafka.bootstrap.servers", "localhost:29092") \
   .option("subscribe", get_input_topic_name()) \
   .option("startingOffsets", "EARLIEST") \
   .load()
```

3. Read the input data:

```
input_visits = input_data.selectExpr('CAST(value AS STRING)', 'timestamp') \
   .select(F.from_json(F.col('value'), "userId INT, eventTime TIMESTAMP, browserKey
STRING, browserVersion STRING, visitedPage STRING")
           .alias('data'), 'timestamp') \
   .selectExpr('data.*', 'timestamp')
```

** I'm using the Apache Kafka append time in my idempotency strategy; hence you can see the value and timestamp in the deserialized columns **

4. Keep the filtering logic and prepare the dataset for the stateful transformation:

```
valid_visits = input_visits.filter('visitedPage IS NOT NULL AND eventTime IS NOT
NULL')
user_visits = valid_visits.withWatermark('eventTime', '15 minutes') \
   .groupBy(F.col("userId"))
```

5. Call your stateful mapping function:

```
user_sessions = user_visits.applyInPandasWithState(
   func=generate_visit_output,
   outputStructType=StructType([
       StructField("sessionId", StringType()),
       StructField("userId", IntegerType()),
       StructField("startTime", TimestampType()),
       StructField("endTime", TimestampType()),
       StructField("browserCode", StringType()),
       StructField("browserVersion", StringType()),
       StructField("navigation", ArrayType(StructType([
           StructField("page", StringType()), StructField("timeSpent", LongType())
       ]))),
   ]),
   stateStructType=StructType([
       StructField("firstEventTimeEpochMillis", LongType()),
       StructField("pages", ArrayType(StructType([
           StructField("visitedPage", StringType()),
StructField("eventTimeAsMilliseconds", LongType())
       ]))),
       StructField("browserCode", StringType()),
```

```
        StructField("browserVersion", StringType())
    ]),
    outputMode="update",
    timeoutConf="EventTimeTimeout"
)
```

6. Read the reference dataset for the browser information:

```
technical_reference_dataset = spark_session.read.schema('id STRING, name STRING,
version STRING') \
    .json(get_reference_dataset_location())
```

7. Process the userSessions by filtering out the empty rows and joining them with the reference dataset. You can also prepare the final output at this moment by using the TO_JSON function:

```
session_to_write = user_sessions.join(technical_reference_dataset,
                                      F.expr('id = browserCode AND version =
browserVersion'), 'left_outer') \
    .withColumn('technicalContext', F.struct('name', 'version')) \
    .drop('browserCode', 'browserVersion', 'id', 'name', 'version') \
    .selectExpr('TO_JSON(STRUCT(*)) AS value')
```

8. Write the transformation result to the output Kafka topic:

```
write_query = session_to_write.writeStream.outputMode("update") \
    .option('checkpointLocation', "/tmp/bde/6/e1/checkpoint/stateful/") \
    .format('kafka')\
    .option('kafka.bootstrap.servers', 'localhost:29092') \
    .option('topic', get_output_topic_name()) \
    .start()
```

Exercise 2 - partial results trigger

Scala/Java

1. Create a new class that extends Trigger<Object, TimeWindow>

```
public class PartialEventTimeWindowTrigger extends Trigger<Object, TimeWindow> {
```

2. You'll be asked to define the methods of the parent class:

```
@Override
public TriggerResult onElement(Object element, long timestamp, TimeWindow window,
TriggerContext ctx) throws Exception {
   return null;
}

@Override
public TriggerResult onProcessingTime(long time, TimeWindow window, TriggerContext
ctx) throws Exception {
   return null;
}

@Override
public TriggerResult onEventTime(long time, TimeWindow window, TriggerContext ctx)
throws Exception {
   return null;
}

@Override
public void clear(TimeWindow window, TriggerContext ctx) throws Exception {

}
```

3. You can implement each of them as a passthrough function returning TriggerResult.CONTINUE or initialize another trigger inside the class and delegate the execution to it.

```
private final EventTimeTrigger eventTimeTrigger = EventTimeTrigger.create();

@Override
public TriggerResult onProcessingTime(long watermarkTime, TimeWindow window,
TriggerContext triggerContext) throws Exception {
   return eventTimeTrigger.onProcessingTime(watermarkTime, window, triggerContext);
}

@Override
public TriggerResult onEventTime(long watermarkTime, TimeWindow window,
TriggerContext triggerContext) {
   LOG.info("Triggering on the event time");
   return eventTimeTrigger.onEventTime(watermarkTime, window, triggerContext);
}

@Override
public void clear(TimeWindow window, TriggerContext triggerContext) throws Exception
{
   eventTimeTrigger.clear(window, triggerContext);
}
```

These methods are irrelevant for our homework. You should focus on the onElement function that is called for each processed element individually.

4. Define the partial results emission logic by using the event time from the input log and the current watermark from the method signature:

```
private static final Logger LOG =
LoggerFactory.getLogger(PartialEventTimeWindowTrigger.class);

private static final long PARTIAL_RESULTS_MILLIS_THRESHOLD =
TimeUnit.MINUTES.toMillis(3L);

private final EventTimeTrigger eventTimeTrigger = EventTimeTrigger.create();

@Override
public TriggerResult onElement(Object element, long watermarkTime, TimeWindow
window, TriggerContext triggerContext) throws Exception {
   // let's check now what are the partial results we should emit
   long remainingMillisBeforeWindowEnd = window.maxTimestamp() - watermarkTime;
   if (remainingMillisBeforeWindowEnd <= PARTIAL_RESULTS_MILLIS_THRESHOLD) {
       LOG.info("Generating partial results because the difference is
"+remainingMillisBeforeWindowEnd+ " ms");
       return TriggerResult.FIRE;
   }
   return eventTimeTrigger.onElement(element, watermarkTime, window,
triggerContext);
}
```

5. Call the created trigger in the job:

```
WindowedStream<Visit, String, TimeWindow> tumblingVisitWindow =
visits.keyBy(Visit::getBrowser)
   .window(
           TumblingEventTimeWindows.of(Time.minutes(5))
   ).allowedLateness(
           Time.seconds(0)
   ).trigger(
           new PartialEventTimeWindowTrigger()
   );
```

Python

1. Create a new class that extends Trigger

```
class PartialEventTimeWindowTrigger(Trigger):
```

2. You'll be asked to define the methods of the parent class:

```
def on_element(self, element: T, timestamp: int, window: W, ctx:
'Trigger.TriggerContext') -> TriggerResult:
   pass

def on_processing_time(self, time: int, window: W, ctx: 'Trigger.TriggerContext') ->
TriggerResult:
   pass
```

```
def on_event_time(self, time: int, window: W, ctx: 'Trigger.TriggerContext') ->
TriggerResult:
   pass

def on_merge(self, window: W, ctx: 'Trigger.OnMergeContext') -> None:
   pass

def clear(self, window: W, ctx: 'Trigger.TriggerContext') -> None:
   pass
```

3. You can implement each of them as a passthrough function returning TriggerResult.CONTINUE or change your trigger so that it implements EventTimTrigger.

```
class PartialEventTimeWindowTrigger(EventTimeTrigger):
```

The single relevant method for the homework is the onElement.

4. Define the partial results emission logic by using the event time from the input log and the current watermark from the method signature:

```
PARTIAL_RESULTS_MILLIS_THRESHOLD_3_MIN_AS_MILLIS = 3 * 60 * 1000

def on_element(self, element: T, watermark_time: int, window: W, ctx:
'Trigger.TriggerContext') -> TriggerResult:
   remaining_millis_before_window_end = (window.max_timestamp() - watermark_time)
   if remaining_millis_before_window_end <=
PartialEventTimeWindowTrigger.PARTIAL_RESULTS_MILLIS_THRESHOLD_3_MIN_AS_MILLIS:
       print("Emitting partial results")
       return TriggerResult.FIRE
   else:
       return super().on_element(element, watermark_time, window, ctx)
```

5. Call the created trigger in the job:

```
records_per_visit_counts: WindowedStream = data_source.map(map_json_to_visit) \
   .key_by(lambda visit:
visit.browser).window(TumblingEventTimeWindows.of(Time.minutes(5))) \
   .allowed_lateness(0) \
   .trigger(PartialEventTimeWindowTrigger())
```

Exercise 3 - window-based counters

Scala/Java

1. First, set the timezone to UTC to avoid any discrepancies between the real data and the display:

```
val sparkSession = SparkSession.builder().master("local[*]")
 .config("spark.sql.session.timeZone", "UTC")
 .getOrCreate()
TimeZone.setDefault(TimeZone.getTimeZone("UTC"))
```

2. Read the input data:

```
val inputStream = sparkSession.readStream.format("kafka")
 .options(Map(
   "kafka.bootstrap.servers" -> "localhost:29092",
   "subscribe" -> InputTopic,
   "startingOffsets" -> "EARLIEST",
 )).load()
```

3. Prepare the input data for processing by mapping the raw JSON into columns:

```
val windowsWithBrowserCount = inputStream
 .select(functions.from_json(
   $"value".cast("string"), "eventTime TIMESTAMP, browser STRING",
      Map.empty[String, String])
   .as("value"))
 .selectExpr("value.*")
```

4. Add the stateful part with the watermark declaration:

```
.withWatermark("eventTime", "5 minutes")
```

5. Define the grouping logic:

```
.groupBy($"browser", functions.window($"eventTime", "5 minutes"))
.agg("eventTime" -> "max", "browser" -> "count")
```

6. Define the output structure:

```
.selectExpr("browser", "window",
 "`count(browser)` AS count",
 "CAST(window.end - `max(eventTime)` AS LONG) AS completeness")
```

7. Write the output to the Apache Kafka topic. Use the TO_JSON function to convert all columns:

```
val writeQuery = windowsWithBrowserCount
 .selectExpr("TO_JSON(STRUCT(*)) AS value")
 .writeStream.outputMode("update")
 .format("kafka")
 .options(Map(
   "checkpointLocation" -> "/tmp/bde/6/e3/checkpoint",
   "kafka.bootstrap.servers" -> "localhost:29092",
   "topic"-> OutputTopic
 )).start()
```

Python

1. First, include the Apache Kafka consumer in the loaded packages:

```
spark_session = SparkSession.builder.master("local[*]")\
   .config("spark.sql.session.timeZone", "UTC")\
   .config('spark.jars.packages',
       'org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0')\
   .config("spark.sql.shuffle.partitions", 2).getOrCreate()
```

2. Read the input data:

```
input_data = spark_session.readStream.format("kafka") \
   .option("kafka.bootstrap.servers", "localhost:29092")\
   .option("subscribe", get_input_topic_name())\
   .option("startingOffsets", "EARLIEST")\
   .load()
```

3. Prepare the input data for processing by mapping the raw JSON into columns:

```
windows_with_browser_groups = input_data\
   .select(F.from_json(F.col('value').cast('string'), 'eventTime TIMESTAMP, browser
STRING').alias('value'))\
   .selectExpr('value.*')\
```

4. Add the stateful part with the watermark declaration:

```
.withWatermark('eventTime', '5 minutes')\
```

5. Define the grouping logic:

```
.groupBy('browser', F.window(F.col("eventTime"), "5 minutes"))
```

6. Define the aggregation for the groups:

```
windows_with_browser_aggregation = windows_with_browser_groups.agg(
   F.max('eventTime'), F.count('browser')
)
```

7. Select the columns you want to write:

```
output_dataframe = windows_with_browser_aggregation.selectExpr('browser', 'window',
               '`count(browser)` AS count',
               'CAST(window.end - `max(eventTime)` AS LONG) AS completeness')
```

8. Write the output to the Apache Kafka topic. Use the TO_JSON function to convert all columns:

```
write_query = output_dataframe.selectExpr('TO_JSON(STRUCT(*)) AS value')\
   .writeStream.outputMode('update')\
   .format('kafka')\
   .option('checkpointLocation', '/tmp/bde/6/e3/checkpoint')\
   .option('kafka.bootstrap.servers', 'localhost:29092')\
   .option('topic', get_output_topic_name())
```

Hope it helps 🙏
Bartosz.